

DeepCampus — Proje Rehberi ve Kod Haritası

Bu doküman projeyle çalışacak kişi (ve code review yapacak hoca) için hazırlanmıştır. Hangi dosya ne işe yarar, özellikler nerede yaşar, sistem nasıl çalıştırılır, veri nasıl akar — hepsi adım adım anlatılır.

Genel İngilizce tanıtım ve özellik listesi için ayrıca README.md dosyasına bakılabilir. Bu dosya ise "kodun neresinde ne var ve nasıl çalıştırırım" sorusuna odaklanır.

1. Proje Tek Cümlede

DeepCampus, akademik PDF'ler üzerinde soru-cevap yapan, tamamen yerel (local-first) çalışan bir Multimodal Hybrid RAG sistemidir: PDF içindeki metin + tablo + figürleri anlar, hem anlamsal (dense) hem kelime (sparse) araması yapar, yerel bir LLM ile kaynak göstererek cevap üretir.

Bileşen özeti:

Katman	Teknoloji	Görev
Arayüz	Streamlit	Sohbet, PDF yükleme, ses, tema
Backend	FastAPI	API, RAG pipeline, JWT doğrulama
Kimlik	Keycloak (OIDC)	Giriş, kullanıcı/rol yönetimi
Vektör DB	Qdrant	dense + sparse vektör arama
LLM	Ollama	Yerel dil modeli çalıştırma
PDF Parse	Docling	Layout + tablo (TableFormer) + figür
Embedding	BGE-M3	Metni dense + sparse vektöre çevirme
Görsel	Moondream2	PDF figür/tablolara ilişkin metin özeti
OCR (yedek)	EasyOCR	Docling açamazsa taranmış sayfa OCR'ı

2. Tam Dosya Yapısı

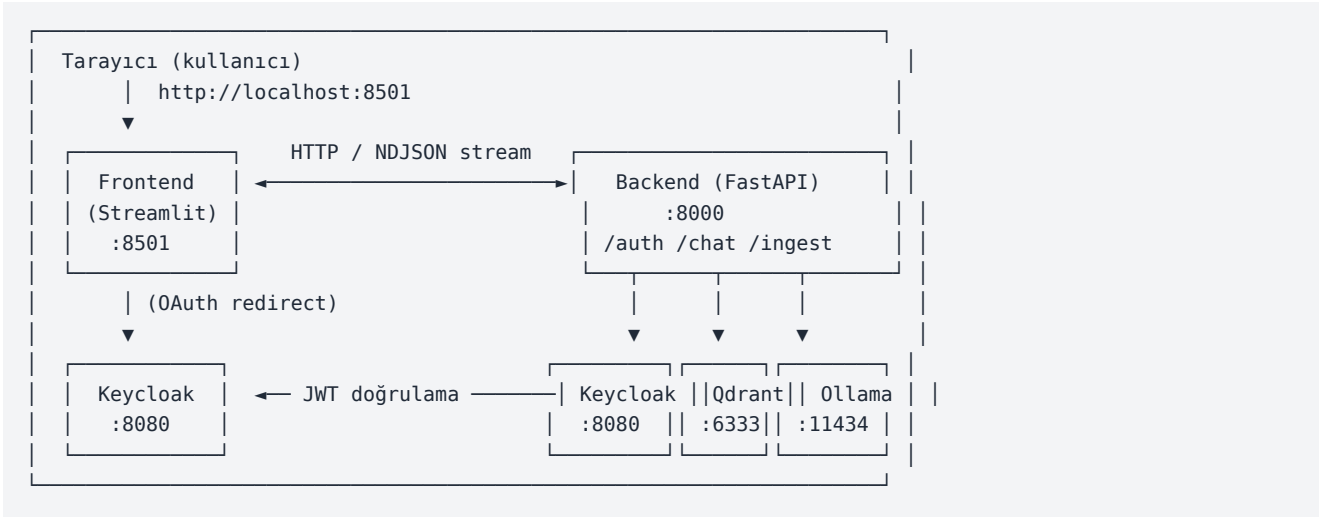
```

Multimodal-RAG-Project/
├── docker-compose.yml      # 5 servisi (keycloak, qdrant, ollama, backend, frontend) ayağa kaldırır
├── .env.example            # Örnek ortam değişkenleri (kopyalayıp .env yapılır – opsiyonel)
├── requirements.backend.txt # Backend Python bağımlılıkları (torch, docling, FlagEmbedding...)
├── requirements.frontend.txt # Frontend Python bağımlılıkları (streamlit < 1.57 ...)
├── README.md              # İngilizce genel tanıtım + özellik listesi
├── PROJE_REHBERI.md       # (bu dosya) dosya haritası + çalışma rehberi
├── docker/
│   ├── Dockerfile.backend # Backend imajı (Python 3.12 + CUDA torch + BuildKit pip cache)
│   ├── Dockerfile.frontend # Frontend imajı (Streamlit)
│   └── keycloak/
│       └── realm-deepcampus.json # Keycloak realm tanımı (client, roller, admin kullanıcı)
├── prompts/
│   └── rag_answer.txt       # LLM'e verilen RAG cevap şablonu (talimatlar + bağlam + soru)
├── backend/
│   └── # FastAPI uygulaması
│       ├── main.py         # Giriş noktası: router'ları bağlar, başlangıçta tablo+koleksiyon hazırlar
│       ├── security.py     # JWT doğrulama dependency'leri (get_current_user, require_instructor)
│       ├── schemas.py      # Pydantic istek/cevap modelleri
│       └── routers/
│           ├── auth.py     # login (password) + OAuth Code flow + register
│           ├── chat.py     # Topics CRUD, history, streaming RAG sorgu, model listesi, benchmark
│           └── ingest.py   # PDF upload, ingestion çalıştır, durum, görsel servis, reset
├── services/
│   └── # Backend ile paylaşılan iş mantığı (ingestion bunları subprocess'te de kullanır)
│       ├── config.py       # Merkezi ayar (dataclass'lar + env okuma)
│       ├── auth.py         # Sohbet session (Topics) + mesaj geçmişi SQLite persistence
│       ├── keycloak_auth.py # Keycloak OIDC: login, code exchange, JWT doğrulama, admin API
│       ├── logging_config.py # Ortak logger
│       ├── pdf_extractor.py # Docling sarmalayıcı (GPU'lu, metin + figür/tablo PNG çıkarır)
│       ├── pdf_fingerprint.py # Tekrar yükleme tespiti (dosya hash + içerik hash)
│       ├── ingestion.py    # ANA pipeline: PDF -> parse -> görsel özet -> chunk -> embed -> Qdrant
│       ├── embeddings.py   # BGE-M3 sarmalayıcı (dense + sparse üretir)
│       ├── vectorstore.py  # Qdrant istemcisi (koleksiyon, batch upsert, dedup)
│       ├── retriever.py    # Hibrit arama + LLM bağlamı oluşturma
│       ├── fusion.py       # Weighted RRF (dense + sparse sonuçlarını birleştirir)
│       └── llm.py          # Ollama istemcisi (model listele/pull, chat stream, benchmark)
├── frontend/
│   └── # Streamlit uygulaması
│       ├── app.py         # Ana uygulama (giriş, sidebar, Topics, sohbet, streaming render)
│       ├── api_client.py  # Backend'e HTTP istemcisi (tüm endpoint çağrılarını)
│       ├── session.py     # localStorage tabanlı oturum kalıcılığı (F5'e dayanıklı)
│       ├── components.py  # Tekrar kullanılan UI parçaları (hero, kart, rozet vs.)
│       ├── styles.py      # CSS tema enjeksiyonu + küçük JS yardımcılarını
│       └── .streamlit/
│           └── config.toml # Streamlit tema/sunucu ayarları
├── tests/
│   └── # pytest testleri
│       ├── conftest.py    # Test fixture'ları (path'leri geçici dizine yönlendirir)
│       ├── test_config.py # Ayar yükleme + env override testleri
│       ├── test_sanitize.py # Dosya adı temizleme (güvenlik) testleri
│       ├── test_retriever.py # RRF / context oluşturma testleri
│       └── test_fingerprint.py # PDF dedup fingerprint testleri
└── (çalışma anında oluşan, git'e girmeyen klasörler)
    ├── docs/              # Yüklenen PDF'ler
    ├── docs_images/       # Çıkarılan figür/tablo PNG'leri
    └── data/              # user.db (sohbet geçmişi), ingest_state.json, image_summaries.json

```

3. Mimari ve Servisler

5 servis Docker Compose ile çalışır:



Her servisin port ve görevi:

Servis	Port	Konteyner adı	Görev
frontend	8501	deepcampus_frontend	Streamlit arayüzü
backend	8000	deepcampus_backend	API + RAG + ingestion (GPU)
keycloak	8080	deepcampus_keycloak	Kimlik doğrulama
qdrant	6333/6334	deepcampus_qdrant	Vektör veritabanı
ollama	11434	deepcampus_ollama	LLM motoru (GPU)

GPU notu: Sadece backend ve ollama GPU kullanır (docker-compose'da deploy.resources.reservations.devices ile NVIDIA passthrough). Docling, BGE-M3, Moondream2 backend içinde; LLM'ler ollama içinde GPU'da çalışır.

4. Kod Haritası — "Hangi özellik nerede?"

Code review'da "şu özelliğin kodu nerede" diye ararsan:

Özellik	Ana dosya(lar)
Giriş (OAuth Code flow)	services/keycloak_auth.py (build_login_url, exchange_code) + backend/routers/auth.py + frontend/app.py (_handle_oauth_callback)
Giriş (password grant, yedek)	services/keycloak_auth.py (login) + backend/routers/auth.py
JWT doğrulama	services/keycloak_auth.py (verify_token, JWKS) + backend/security.py
Kullanıcı kaydı	services/keycloak_auth.py (create_user, Admin API)
Çoklu sohbet (Topics)	services/auth.py (chat_sessions tablosu, CRUD) + backend/routers/chat.py (/sessions) + frontend/app.py (sidebar Topics)
Sohbet geçmişi	services/auth.py (chat_history) + backend/routers/chat.py (/history)

Özellik	Ana dosya(lar)
F5'e dayanıklı oturum	frontend/session.py (localStorage)
PDF yükleme	backend/routers/ingest.py (/upload) + frontend/app.py (file_uploader)
PDF parse (Docling)	services/pdf_extractor.py
PDF parse (yedek)	services/ingestion.py (_extract_pdf_pymupdf)
Tekrar tespiti (dedup)	services/pdf_fingerprint.py
Görsel özetleme (VLM)	services/ingestion.py (_summarize_image, Moondream2)
Chunking	services/ingestion.py (RecursiveCharacterTextSplitter)
Embedding (BGE-M3)	services/embeddings.py
Vektör saklama	services/vectorstore.py (Qdrant upsert)
Hibrit arama	services/retriever.py (dense + sparse)
RRF birleştirme	services/fusion.py
LLM cevap (stream)	services/llm.py (stream_chat) + backend/routers/chat.py (/query)
RAG prompt şablonu	prompts/rag_answer.txt
Model seçimi/indirme	services/llm.py + backend/routers/chat.py (/models)
Streaming render	frontend/app.py (stream_query döngüsü)
Tema (dark/light)	frontend/styles.py (inject_styles) + frontend/app.py (_toggle_theme)
Ses (STT/TTS)	frontend/app.py (_speak_button, speech_to_text)
Ayarlar	services/config.py

5. Veri Akışları

5.1. Ingestion (PDF işleme) akışı

Tetikleyici: arayüzde Process & update database butonu → POST /ingest/run → services/ingestion.py içindeki run_ingestion().

docs/ klasöründeki her PDF için:

1. Fingerprint hesaplama (dosya hash + içerik hash) [pdf_fingerprint.py]
2. ingest_state.json + Qdrant'ta var mı? Varsa ATLA (dedup)
3. _extract_pdf: [ingestion.py]
 - a. Önce Docling dene [pdf_extractor.py]
 - layout-aware metin (okuma sırası)
 - TableFormer ile tablo PNG
 - layout dedektör ile figür PNG
 - built-in OCR (taranmış sayfa)
 - b. Docling patlarsa PyMuPDF + EasyOCR fallback [ingestion.py]
4. Her figür/tablo PNG'sini Moondream2 ile özetle [ingestion.py]
 - "[IMAGE SUMMARY]: ..." metni üret
 - data/image_summaries.json'a da yaz
5. VLM'i bellekten boşalt, BGE-M3 yükle (sıralı VRAM)
6. Chunk'la (1024 token / 128 overlap, BGE-M3 tokenizer) [ingestion.py]
7. Embed et (dense 1024-d + sparse) [embeddings.py]
8. Qdrant'a 64'lük batch'lerle upsert [vectorstore.py]
9. ingest_state.json güncelle

5.2. Sorgu (RAG) akışı

Tetikleyici: sohbet kutusuna soru → POST /chat/query → backend/routers/chat.py içindeki query_chat().

- | | |
|---|--------------------------|
| 1. session_id'yi çöz (yoksa General Chat) | [auth.py] |
| 2. Kullanıcı mesajını kaydet | |
| 3. Soruyu embed et (dense + sparse) | [embeddings.py] |
| 4. Qdrant'ta İKİ arama: dense + sparse | [retriever.py] |
| 5. RRF ile birleştir (0.6 dense / 0.4 sparse, k=60) | [fusion.py] |
| 6. En iyi chunk'lardan bağlam + görsel + kaynak çıkar | [retriever.py] |
| 7. prompt şablonunu doldur (geçmiş + bağlam + soru) | [llm.py, rag_answer.txt] |
| 8. Ollama'dan cevabı TOKEN TOKEN stream et | [llm.py] |
| 9. NDJSON event'leri frontend'e gönder: | |
| {event: session} {event: sources} {event: token}... | |
| {event: images} {event: done} | |
| 10. Cevabı + kaynakları + görselleri kaydet | |

5.3. Giriş (OAuth Authorization Code) akışı

- | | |
|--|----------------------------------|
| 1. "Sign in with Keycloak" linkine tıkla | [frontend/app.py] |
| 2. Frontend backend'den login URL ister | [api_client.py] |
| GET /auth/login-url?redirect_uri=... | [routers/auth.py] |
| 3. Tarayıcı Keycloak'a yönlendir (public_url) | [keycloak_auth.py] |
| 4. Kullanıcı Keycloak'ın KENDİ sayfasında parolayı girer | |
| 5. Keycloak ?code=... ile frontend'e geri yönlendirir | |
| 6. Frontend code'u backend'e gönderir | [app.py: _handle_oauth_callback] |
| POST /auth/exchange-code | [routers/auth.py] |
| 7. Backend code'u token'a çevirir (server-to-server) | [keycloak_auth.py] |
| 8. Token + id_token + role localStorage'a yazılır | [session.py] |

6. Çalıştırma Rehberi (Adım Adım)

6.1. Ön gereksinimler

- Docker Desktop (Windows: WSL2 backend açık)
- NVIDIA GPU + güncel sürücü (önerilen 8 GB+ VRAM; 6 GB ile q4 modeller)
- NVIDIA Container Toolkit (Docker Desktop'ta WSL2 ile gelir)
- Git
- Chromium tabanlı tarayıcı (ses özellikleri için)

6.2. GPU'nun Docker'da çalıştığını doğrula

```
docker run --rm --gpus all nvidia/cuda:12.1.0-base-ubuntu22.04 nvidia-smi
```

GPU tablosu gelirse tamam. Çıktıdaki VRAM miktarını not et (model seçimini etkiler).

6.3. Projeyi çek

```
git clone https://github.com/mustafayazbahar/Multimodal-RAG-Project.git
cd Multimodal-RAG-Project
```

ZIP indirme YERINE git clone kullan — güncellemeleri git pull ile alabilirsin, her seferinde ZIP indirmen gerekmez.

6.4. (Opsiyonel) .env oluştur

.env zorunlu değildir; docker-compose her değişken için varsayılan tutar. Ama 6 GB VRAM'de modelleri küçültmek için gerekir:

```
# 6 GB GPU için (RTX 3060 Laptop, 2060, 2070 vb.)
LLM_MODEL=llama3.1:8b-instruct-q4_K_M
AVAILABLE_LLMS=llama3.1:8b-instruct-q4_K_M,qwen2.5:7b-instruct-q4_K_M,gemma2:9b-instruct-q4_K_M
```

12 GB+ VRAM'de default'lar (Qwen 14B dahil) çalışır, .env gerekmez.

6.5. Stack'i başlat

```
docker compose up -d --build
```

- İlk build 15-25 dk (PyTorch CUDA + BGE-M3 iner).
- Sonraki build'ler BuildKit pip cache ile 2-5 dk.
- Durumu izle: docker compose ps (hepsi healthy olmalı).

6.6. CUDA'nın konteynerde çalıştığını doğrula

```
docker exec deepcampus_backend python -c "import torch; print('CUDA:', torch.cuda.is_available())"
```

CUDA: True çıkmalı.

6.7. LLM modellerini indir

```
docker exec -it deepcampus_ollama ollama pull llama3.1:8b-instruct-q4_K_M
docker exec -it deepcampus_ollama ollama pull qwen2.5:7b-instruct-q4_K_M
```

Veya arayüzden: sidebar → Download more models.

6.8. Arayüzleri aç

URL	Ne
http://localhost:8501	Ana uygulama
http://localhost:8000/docs	Backend API (Swagger)
http://localhost:6333/dashboard	Qdrant paneli
http://localhost:8080	Keycloak admin

Varsayılan giriş: admin / admin123 (instructor rolü).

7. Yaygın Görevler

PDF ekleyip indeksleme

1. Giriş yap (instructor).
2. Sidebar → Knowledge base → Upload PDF ile dosyayı yükle.
3. Process & update database butonuna bas.
4. İlk run'da Docling modelleri iner (~1.5 GB), bekle.
5. İlerlemeyi izle: docker compose logs -f backend.

Yeni konu (Topic) açma

Sidebar → My Topics → Create new topic → isim ver. Her konu ayrı sohbet geçmişi tutar; "General Chat" silinemez.

Aktif modeli değiştirme

Sidebar → Model → Active LLM açılır menüsünden seç.

Bilgi tabanını sıfırlama

Sidebar → Danger zone → Reset knowledge base. Qdrant koleksiyonunu + state dosyasını siler. (Sadece PDF'i docs/'tan silmek yetmez, fingerprint Qdrant'ta kalır.)

Kod değişikliğini yansıtma

Kod imaja build sırasında kopyalanır (volume mount değil). Değişiklik için:

```
docker compose up -d --build backend      # veya frontend
```

Sadece restart yetmez.

Durdur / devam ettir

```
docker compose stop      # durdur, veri (volume) durur
docker compose start     # geri getir
docker compose down      # konteynerleri kaldır (volume'lar -v olmadan kalır)
```

8. Code Review İçin Öne Çıkan Teknik Kararlar

Hocanın dikkatini çekebilecek, bilinçli alınmış kararlar:

1. Hibrit retrieval (dense + sparse) tek modelden. BGE-M3 tek geçişte ikisini birden üretir (embeddings.py), ayrı bir BM25 altyapısı kurmadan hybrid arama yapılır. Birleştirme Weighted RRF ile (fusion.py).
2. Docling primary + PyMuPDF fallback. Vektör tablolar/figürler PyMuPDF ile kaçıyordu; Docling (TableFormer) bunları yakalar. Docling patlarsa try/except ile eski yola düşülür, PDF yine indekslenir (ingestion.py).

3. Sıralı VRAM yönetimi. Tek GPU'da Moondream2 → boşalt → BGE-M3 → boşalt şeklinde sıralı yükleme; eşzamanlı OOM önlenir (ingestion.py).
4. Çok katmanlı dedup. Dosya hash'i (byte birebir) + içerik hash'i (ilk sayfalar + başlık/yazar). Sadece başlık eşleşmesi bilerek dedup sebebi sayılmaz — "Ders Notları" gibi ortak başlıklar yanlış pozitif veriyordu (pdf_fingerprint.py).
5. Keycloak'a auth devri. Eski SQLite+bcrypt yerine OIDC. Parola, OAuth Code flow'da Keycloak'ın sayfasında girilir, Streamlit'ten geçmez. JWT realm JWKS'ine karşı RS256 doğrulanır (keycloak_auth.py, security.py).
6. İki Keycloak URL'i. İç ağ (url, server-to-server) vs tarayıcı (public_url, OAuth redirect). Tarayıcı keycloak:8080'i çözemez (config.py, keycloak_auth.py).
7. localStorage oturum (URL'de token yok). F5'e dayanıklı; token URL'de tutulmaz (Referer/log sızıntısı önlenir) (session.py).
8. Batch Qdrant upsert. Büyük ders kitaplarında tek seferde upsert timeout veriyordu; 64'lük batch + uzatılmış client timeout (vectorstore.py).
9. GPU'lu Docling. accelerator_options ile CUDA/MPS/CPU otomatik seçimi; CPU'da 10x+ yavaş olan parsing GPU'da hızlanır (pdf_extractor.py).
10. Streamlit < 1.57 pin'i. st.components.v1.html 1.56'da deprecate edildi; TTS/scroll/autofocus bu fonksiyona dayandığı için sürüm sabitlendi (requirements.frontend.txt).

9. Sorun Giderme

Belirti	Sebebi	Çözüm
torch.cuda.is_available()=False	GPU passthrough yok	NVIDIA Container Toolkit / Docker restart
CUDA out of memory (ingestion)	Docling+Moondream+LLM çakıştı	docker compose restart ollama sonra ingest
CUDA out of memory (cevap)	q8/14B model 6 GB'de	.env'de q4 modele geç
container name already in use	Eski konteyner kaldı	docker compose down sonra up -d
Ingestion başlamıyor	Upload ≠ Process	Process & update database butonuna bas
no space left on device	WSL2 disk doldu	docker system prune -a (modeller de gider)
Sign in siyah ekran/takılma	Eski kod	git pull ile güncelle
Kod değişti yansımıyor	Sadece restart yapıldı	docker compose up -d --build <servis>
GPU %0, ingestion yavaş	İş gerçekten başlamadı	Log'da POST /ingest/run var mı kontrol et

10. Hızlı Komut Referansı


```
# Başlat / build
docker compose up -d --build
docker compose --progress=plain up -d --build # canlı build çıktısı

# Durum & log
docker compose ps
docker compose logs -f backend # canlı backend log
docker compose logs backend | grep -i docling # sadece Docling satırları

# GPU izleme (ayrı terminal)
nvidia-smi -l 2

# Konteyner içine bak
docker exec deepcampus_backend python -c "import torch; print(torch.cuda.is_available())"
docker exec -it deepcampus_ollama ollama list

# Model indir
docker exec -it deepcampus_ollama ollama pull qwen2.5:7b-instruct-q4_K_M

# Testler
docker exec deepcampus_backend python -m pytest tests/ -v

# Durdur / temizle
docker compose stop
docker compose down
docker system prune -a # tüm kullanılmayan imaj/cache (dikkat)
```

11. Ortam Değişkenleri (en önemlileri)

Değişken	Varsayılan	Açıklama
LLM_MODEL	llama3.1:8b-instruct-q8_0	Varsayılan model
AVAILABLE_LLMS	3 modellik liste	Arayüz menüsü
EMBEDDING_MODEL	BAAI/bge-m3	Embedding modeli
CHUNK_SIZE	1024	Chunk token boyutu
CHUNK_OVERLAP	128	Chunk'lar arası örtüşme
TOP_K	20	RRF öncesi aday sayısı
RERANK_TOP_N	8	LLM'e giden chunk sayısı
DENSE_WEIGHT / SPARSE_WEIGHT	0.6 / 0.4	RRF ağırlıkları
KEYCLOAK_URL	http://keycloak:8080	İç ağ Keycloak (server-to-server)
KEYCLOAK_PUBLIC_URL	http://localhost:8080	Tarayıcı Keycloak (OAuth redirect)
FRONTEND_URL	http://localhost:8501	OAuth redirect_uri
TEMPERATURE	0.3	LLM yaratıcılığı

DeepCampus v2.5 · Proje Rehberi · Hazırlanma: 2026